Image Processing for Mechatronics MCTR1010
Mechatronics Engineering
Faculty of Engineering and Materials Science
German University in Cairo

Goalkeeper Robotic Arm

By

Team 21

Besher Hezzeni 55-22381
Mohamed Lotfy 55-1400
Mazen Amr 55-0983
Michael Hany 55-23395
Youssef Elkholy 55-1115
Mariam Ahmed 55-4057

Course Team

Assoc. Prof. Dr. Omar M.Shehata

May 5, 2026

Chapter 1

Literature Review

Vision-based robotic systems are widely used in mechatronics for dynamic tasks such as robotic goalkeeping, requiring real-time ball detection and interception. Kim et al. [1] demonstrated an OpenCV-based method using HSV segmentation, contour detection, and centroid extraction for efficient real-time tracking on embedded platforms.

Trajectory prediction improves performance; Liu et al. [2] applied a Kalman filter to estimate motion and predict the ball's path for accurate interception. Deep learning approaches such as YOLOv4 [3] enhance detection robustness, while Sharma et al. [4] used HSV segmentation with morphological operations for efficient low-cost tracking.

Al-Khalifa et al. [5] integrated detection, prediction, and actuation in a robotic goalkeeper, achieving high interception success. Most systems follow a pipeline of image acquisition, preprocessing, feature extraction, and actuation, typically implemented on Raspberry Pi with OpenCV and Arduino.

Overall, combining traditional vision techniques with machine learning enables reliable real-time detection and interception in robotic systems.

Chapter 2

Hardware Design

2.1 Component List

Component Name	Quantity	Approx. Price (EGP)	Purchase Source
Raspberry Pi 4 Model B (4GB)	1	4750	RAM Electronics
Raspberry Pi Camera Module V2	1	800	RAM Electronics
Raspberry Pi Camera Cable	1	185	RAM Electronics
Arduino Uno R3	1	750	RAM Electronics
MicroSD Card 64GB	1	495	RAM Electronics
High Torque Servo Motor (FS5103B)	1	300	RAM Electronics
USB Cable (Arduino Connection)	1	40	RAM Electronics

Table 2.1: Hardware Components Required for the Goalkeeper Robotic Arm

2.2 Project Plan

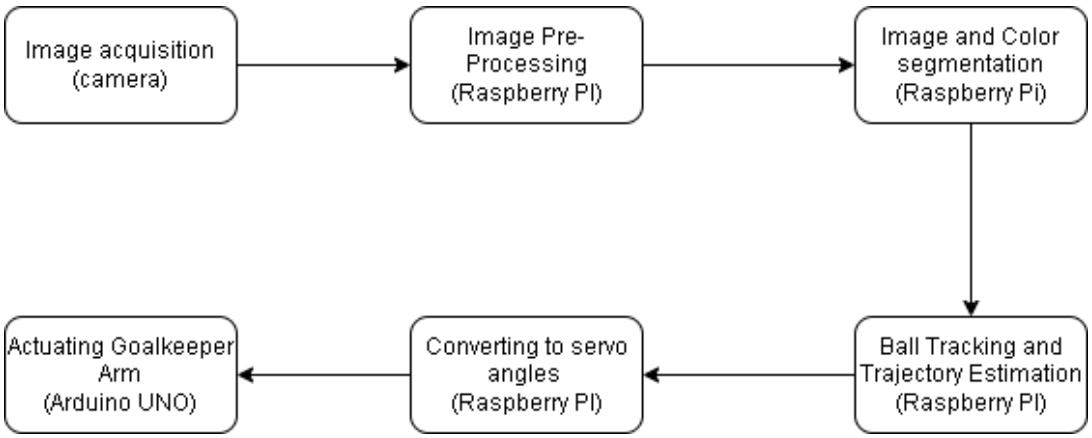

Figure 2.1: Project Plan

Chapter 3

Integrated System Implementation and Results

3.0.1 Overall System Methodology

The system follows a structured pipeline that integrates image processing with real-time hardware control, operating as a closed-loop from image acquisition to motor actuation.

The methodology includes:

- Capturing real-time images using the Raspberry Pi camera
- Applying preprocessing (scaling, rotation, enhancement)
- Performing color segmentation to detect the ball
- Extracting the centroid position
- Mapping the position to predefined zones
- Generating control commands
- Actuating the servo motor via Arduino

This pipeline ensures continuous feedback and enables real-time system response.

3.0.2 Detailed Implementation on Hardware

The system is implemented on the fabricated hardware setup, with the Raspberry Pi acting as the main processing unit for image processing using Python and OpenCV.

The camera continuously captures frames, which are processed to detect the ball and compute its centroid. The detected position is then mapped relative to the frame and transmitted to

the Arduino via serial communication.

The Arduino generates PWM signals to control the servo motor, aligning the robotic arm with the ball position.

This division of tasks allows efficient operation, with the Raspberry Pi handling processing and the Arduino managing real-time actuation.

3.0.3 Experimental Results

Figure **??** illustrates the output of the image processing pipeline. The system successfully captures the image, converts it to HSV for improved color separation, applies thresholding to isolate the ball, and uses morphological operations to reduce noise.

The final result shows accurate ball detection with centroid marking. The grid overlay enables zone classification, and in the example, the ball is correctly identified in Zone 3, generating the appropriate control command.

3.0.4 Hardware Setup and System Operation

The complete hardware setup used in the experiment is shown in Figure 3.1.

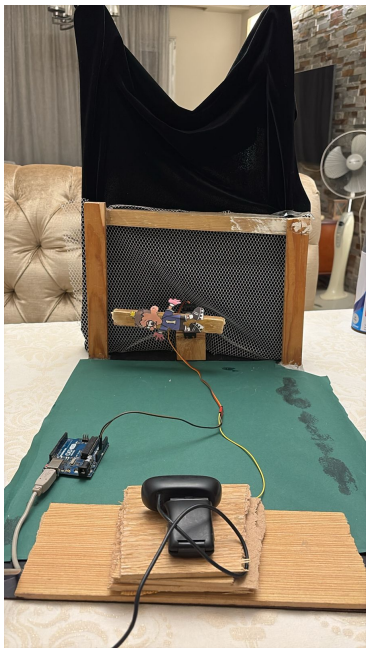

Figure 3.1: Complete Hardware Setup of the Goalkeeper Robotic Arm System

The system operates as follows:

- The camera captures the scene and sends frames to the Raspberry Pi.

- The Raspberry Pi processes each frame and extracts the ball position.
- The computed position is transmitted to the Arduino via USB serial communication.
- The Arduino converts the received signal into PWM signals.
- The servo motor actuates the robotic arm accordingly.

This closed-loop interaction ensures that the robotic arm continuously adjusts its position based on the detected ball location.

3.0.5 Discussion of Results

The system demonstrates reliable ball detection and control signal generation, with accurate centroid localization. HSV color segmentation effectively isolates the object, while morphological operations improve detection by reducing noise.

Some limitations were observed:

- Sensitivity to lighting variations
- Errors from similarly colored objects
- Slight processing delay on the Raspberry Pi

Despite these challenges, the system achieves real-time performance and confirms the feasibility of image-based control on embedded platforms.

3.1 Code Functionality and Processing Logic

The system uses Python and OpenCV to perform real-time ball detection and generate control commands. The camera continuously captures frames, which are processed sequentially.

Each frame is preprocessed (resizing/rotation), converted to HSV, and thresholded to create a binary mask. Morphological operations reduce noise, after which contours are detected and the largest contour is selected. Its centroid is computed and classified into left, center, or right regions.

Based on this classification, commands are sent from the Raspberry Pi to the Arduino via serial communication. This loop runs continuously, enabling real-time responsive actuation.

3.2 Actuation and Real-Time Tracking

Once the ball is detected, its centroid is extracted and used to determine its position in the frame. The image is divided into a 3×3 grid (R1–R9) to classify the ball location.

Based on the detected region, the system generates control commands: left regions rotate the servo left, right regions rotate it right, and the center region maintains position.

The centroid is updated continuously, enabling real-time tracking. Smooth motion follows gradual movement, while rapid movement is handled through discrete zone updates.

The command is sent from the Raspberry Pi to the Arduino, which generates a PWM signal to control the servo motor, forming a closed-loop system for tracking and interception.

3.2.1 Limitations in Actuation

Although the system successfully tracks the ball, some limitations were observed:

- **Discrete Movement:** The use of zone-based control results in step-wise movement rather than smooth continuous tracking.
- **Delay in Response:** Processing time and communication delay introduce slight lag between detection and actuation.
- **Servo Speed Constraints:** The physical speed of the servo motor limits the system's ability to react to very fast ball movements.

Despite these limitations, the system demonstrates effective real-time tracking and actuation based on visual input.

3.3 Implementing Machine Learning

A YOLOv8-based approach was used for real-time object detection on the Raspberry Pi, improving accuracy under varying conditions. The model detects objects and outputs bounding boxes with confidence scores, from which the most relevant object is selected.

The centroid of the detected object is computed as:

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2} \tag{3.1}$$

The frame is divided into a 3×3 grid, and the centroid is mapped to a region. The zone is calculated as:

$$\text{Zone} = (\text{row} \times 3) + \text{column} + 1 \tag{3.2}$$

The zone is sent to the Arduino via serial communication, where it is converted into a PWM signal to control the servo motor. Data is transmitted only when changes occur or after a fixed interval.

This integration enables reliable real-time tracking and control.

Bibliography

[1] S. Kim, J. Lee, and H. Park. Real-time ball tracking using computer vision for robotic applications. *IEEE Access*, 9:11234–11245, 2021.

[2] J. Liu, T. Wang, and Y. Zhang. Vision-based ball trajectory prediction for robotic goalkeepers. *Robotics and Autonomous Systems*, 150:103987, 2022.

[3] A. Bochkovskiy, C. Wang, and H. M. Liao. Yolov4: Optimal speed and accuracy of object detection. *Computer Vision and Pattern Recognition*, 2021.

[4] P. Sharma, R. Gupta, and K. Singh. Color-based object detection and tracking using opencv for mobile robots. *Sensors*, 23(8):4125, 2023.

[5] M. Al-Khalifa, M. Rahman, and S. Ahmed. Design and implementation of a vision-based soccer goalkeeper robot. *IEEE Access*, 10:34567–34578, 2022.